

TestA

Documentation

Lazar Nagulov

January 05, 2026

Contents

Introduction	3
Getting Started	4
Prerequisites	4
Cloning the Project	4
Building the Project	4
Running Tests	4
CLI Usage	5
Generate Command	5
Basic Generation	5
CSV Generation	5
Reproducible Generation	5
Override Generation Count	5
Check Command	5
LSP Command	5
Init Command	5
Info Command	6
Global Flags	6
Features	7
Templates	7
Inheritance	7
Enumerations	8
Data Types	9
Lists	9
Type constraints	10
String Patterns	12
Changelog	13

Introduction

TestA is a domain-specific language designed for generating structured test data in formats like JSON, CSV, and XML. It provides a simple, C-like syntax for defining templates, resources, enums, and generation rules, enabling quick and expressive data mockups for testing, prototyping, or seeding.

```
@output csv {
    header = true;
    quote = true;
    delimiter = ",";
}
@output_path "./example.csv";

type Grade = int[range=6..=10];
type GradeList = [Grade][count=0..=50];

enum Role {
    User => 10;
    Admin => 2;
    Other;
};

template User {
    id = uuid();
    name = string;
    role = Role;
}

template Student : User {
    year = int [range=1..=4];
    grades = GradeList;
}

@generate User [10];
```

Listing 1: testA example

Getting Started

This section describes how to install the necessary tools and set up the project locally.

Prerequisites

To build and run this project, you must have the Rust toolchain installed, which includes the Rust compiler as well as Cargo, Rust's package manager and build system. To install Rust and Cargo, run the following command in your terminal:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

After installation, verify:

```
rustc --version  
cargo --version
```

Cloning the Project

```
git clone https://github.com/lazarnagulov/testa.git  
cd testa
```

Building the Project

```
# From the project root  
cargo install --path . --bins
```

Running Tests

```
cargo test
```

CLI Usage

The TestA CLI provides commands for generating test data, checking syntax, running an LSP server, and managing projects. This section covers the most common use cases and command-line options.

Generate Command

The generate command is the primary way to produce test data from your TestA templates.

Basic Generation

Generate JSON output (default format):

```
testa generate user.testa
testa generate user.testa -o output.json
testa generate user.testa --pretty
```

CSV Generation

Generate CSV output:

```
testa generate user.testa -f csv
testa generate user.testa -f csv -o users.csv
```

Reproducible Generation

Use a seed value to ensure reproducible output:

```
testa generate user.testa --seed 42
```

Override Generation Count

Specify the number of records to generate:

```
testa generate user.testa --count 1000
```

Check Command

Validate TestA files for syntax and semantic errors without generating output.

```
testa check *.testa
testa check user.testa --warnings
testa check user.testa --syntax-only
```

Options:

- --warnings - Show warnings in addition to errors
- --syntax-only - Only check syntax, skip semantic analysis

LSP Command

Start the Language Server Protocol server for editor integration.

```
testa lsp
testa lsp --port 9257
testa lsp --log-file lsp.log
```

Options:

- --port <PORT> - Specify the port for the LSP server
- --log-file <FILE> - Write LSP logs to the specified file

Init Command

Initialize a new TestA project with scaffolding and optional examples.

```
testa init
testa init my-project --with-examples
testa init --name "My Data Project"
```

Options:

- --with-examples - Include example TestA files
- --name <NAME> - Set the project name

Info Command

Display information about the TestA installation and environment.

```
testa info
testa info --extended
```

Options:

- --extended - Show additional diagnostic information

Global Flags

These flags can be used with any command:

```
testa -v generate user.testa      # Verbose output
testa -q generate user.testa      # Quiet mode (minimal output)
testa --version                   # Show version information
testa --help                      # Display help message
```

Available global flags:

- -v, --verbose - Enable verbose output
- -q, --quiet - Suppress non-essential output
- --version - Print version information
- --help - Display help information

Features

Templates

```
template <name> [: <parent_name>] {  
    [override] <name1> = <expr>;  
    ...  
}
```

Listing 7: Template declaration

Templates are blueprints for creating reusable structures that are used in a “generate” directive. Each template has a name and a set of fields, where each field is a pair consisting of a string (the field name) and an expression (which defines the field’s value). By using data types and enumerations within these expressions, you can add randomness to the values generated by the template.

```
template User {  
    id = int;  
    name = string;  
    age = int;  
}
```

Listing 8: Example template in testA

Inheritance

```
template User {  
    age = int [range=0..=100];  
}  
  
template Student : User {  
    year = int[range=1..=4];  
    override age = int[range=19..=30];  
}
```

Listing 9: Inheritance example

Templates can be created from existing templates through a process known as inheritance (Listing 9). A child template inherits all the fields from its parent template and has the option to override any specific field by using the “override” keyword and specifying the field name.

While using this keyword is not mandatory, incorrect usage may lead to a warning. For example, Listing 10 will trigger a warning because the field “age” is not overridden in the child template. It’s important to note that all repeated fields are overridden by default.

```
template User {  
    id = string;  
}  
template Student : User {  
    year = int[range=1..=4];  
    override age = int[range=19..=30];  
}
```

Listing 10: Override example

Enumerations

```
enum <name> {  
    <variant1> [= <expr:int>];  
    ...  
}
```

Listing 11: Enumeration declaration

Enumerations include a list of names and their corresponding weights, which indicate the likelihood of randomly generating a variant. Weights are always integers, with a default value of one if not specified.

```
enum Role {  
    User => 8;  
    Admin;  
    Developer => 3;  
}
```

Listing 12: Enumerations in testA

```
template User {  
    id = int;  
    name = string;  
    role = Role;  
}
```

Listing 13: Template with enumeration field

As shown in listing Listing 13, enumerations can be used as types in templates. Their evaluated values are always represented as strings.

Data Types

Unlike other programming languages, such as C or Java, testA data types represent random values in expressions. There are four fundamental data types available: integers (ints), floating-point numbers (floats), strings, and booleans.

```
template User {  
    name = string;  
    age = int + 18;  
    salary = float + 300.0;  
    is_registered = !bool;  
}
```

Listing 14: Template with data types

As shown in Listing 14, these types can interact with one another as well as with literals. Additionally, fundamental data types can be extended by introducing constraints or creating lists of these types.

Lists

A list is a container that can hold various data types, including other lists. By default, a list generates a random number of items, ranging from 0 to 16. However, this number can be adjusted using the “count” constraint. Further details about constraints can be found in the next section.

```
[int][count=5..15]  
[[int[range=1..5]][count=1..5]][count=3..5]
```

Listing 15: List examples

In Examples from Listing 15, two lists are declared. Second list can be simplified by utilizing type constraints.

Type constraints

```
type <name> = <base_type> [<constraint1>, <constraint2>...];  
type <name> = extend <base_type> with [<constraint1>, ...];
```

Listing 16: Constrained type declaration

The previous example (Listing 14) does not depict a realistic individual, as the fields for age and salary could potentially contain negative values. To address this issue, we can implement type constraints to prevent such errors. These constraints can be easily added by enclosing them in brackets [] after the type name (Listing 17), ensuring that the data conforms to the expected parameters.

```
template User {  
    age = int [range=0..=100];  
}  
  
template Student : User {  
    override age = int [range=19..=100];  
    grades = [int [range=6..=10]] [count=0..=46];  
}
```

Listing 17: Templates with constrained data types

To improve clarity and minimize redundancy, types may be effectively declared and reused (Listing 18 and Listing 19).

```
type Age = int [range=0..=100];  
type StudentAge = extend Age with [min=19];  
type Grade = int [range=6..=10];  
type GradeList = [Grade] [count=1..=46];  
  
template User {  
    age = Age;  
}  
  
template Student : User {  
    override age = StudentAge;  
    grades = GradeList;  
}
```

Listing 18: Templates with user-defined data types

```

type IntList = [int][count=1..=5];
type IntMatrix = [IntList][count=1..=5];
type IntTensor = [IntMatrix][count=1..=5];
// same as
type IntTensor
    = [[[int][count=1..=5]][count=1..=5]][count=1..=5];

```

Listing 19: Templates with user-defined lists

Name	Compatible types	Input	Description
range	int, float	Inclusive or Exclusive range	Sets upper and lower bound.
min	int, float	int	Sets lower bound.
max	int, float	int	Sets upper bound.
multiple_of	int	int	Number is multiple of input.
bias	bool	float [0.0-1.0] or int [0,1]	Sets chance of being true.
count	list	Inclusive / Exclusive range or int	Sets number of items in list.

Table 1: Data type constraints

String Patterns

String patterns define the structure of a string. In these patterns, characters wrapped in `{}` represent placeholders where random values will be generated. Every other character is treated as literal.

Placeholders are:

- `{a}` lowercase letter
- `{A}` uppercase letter
- `{#}` digit

```
string_pattern "pattern: ${aaaAAAA}";
```

Listing 20: String pattern example

Example (Listing 20) can generate “pattern: abcABCD123”

```
template User {  
    id = int;  
    name = string_pattern "${Aa[4]}";  
    email = string_pattern "${a[1..=5]#[0..=4]}@${a[4]}.com"  
}
```

Listing 21: String pattern in template example

Patterns can be utilized directly within the template, as illustrated in Listing 21. They can also be simplified using the repeat syntax [integer or range].

Changelog

v0.2.0 - **Semantic Analysis & Architecture Update**

- Introduced a semantic analyzer
- Refactored core architecture to support semantic validation and extensibility
- Introduced CLI tool

v0.1.0 - **Initial MVP**

- Added CSV template generation
- Implemented enumerations
- Added template inheritance
- Introduced string patterns and attribute support